

IE6-DPL: REPORT LAB 02

RADIX-2 FFT AND REAL-TIME SPECTRUM ANALYSER

Team (Group number + letter as chosen in Moodle)	Team members	
1/1	1	Emir Avni Sahin
	2	Efe Kardes
	3	

Please fill out the document electronically – including the yellow fields above – and upload it to Moodle once per team at the latest 30 min after the end of the lab session (no matter who uploads it, it will automatically be assigned to all members). Please note that the complete assignment can be found in another document.

Lab task 1

1. As a second step, enter the input sequence $x_2[n]$ from prep task and check the result. Do overflows occur? Comment on this and explain the values obtained in a brief calculation.
2. Correct the "error" just determined in the program butterfly.c, so that overflows are avoided. Check the functionality: Are the output values correct?
3. In butterfly.c replace the equations for X2 and Y2 with the equations from the first preparation task. Check that the results remain identical.

- Do overflows occur? Explain the values.

Yes, a hardware overflow occurs. When we tested the input sequence $X_2 = [10000, 0, -10000, 0, 10000, 0, -10000, 0]$, our hand calculations predicted a magnitude of 40000 at bins 2 and 6.

Overflowed values with the original 15 bit right shift:

$$|X(2)| : 652087296, \quad -25536 + j \quad 0$$

$$|X(6)| : 652087296, \quad -25536 + j \quad 0$$

- Correct the missing scaling in the butterfly and give the changed lines of code:

Corrected values after the 16 bit right shift:

$|X(2)|$: 25000000, 5000 + j 0

$|X(6)|$: 25000000, 5000 + j 0

Code changed in FFTbutterfly Modified to >> 16 to divide the audio values by 2 at each stage

```
*p_xa = (int16_t) (lXout_r1_temp>>16);
```

```
*(p_xa+1) = (int16_t) (lXout_i1_temp>>16);
```

```
*p_xb = (int16_t) (lXout_r2_temp>>16);
```

```
*(p_xb+1) = (int16_t) (lXout_i2_temp>>16);
```

- Give the C code for X2 and Y2

```
// Formula: X2 = 2*x1 - X1
```

```
lXout_r2_temp = 2 * ( *p_xa << 15 ) - lXout_r1_temp;
```

```
// Formula: Y2 = 2*y1 - Y1
```

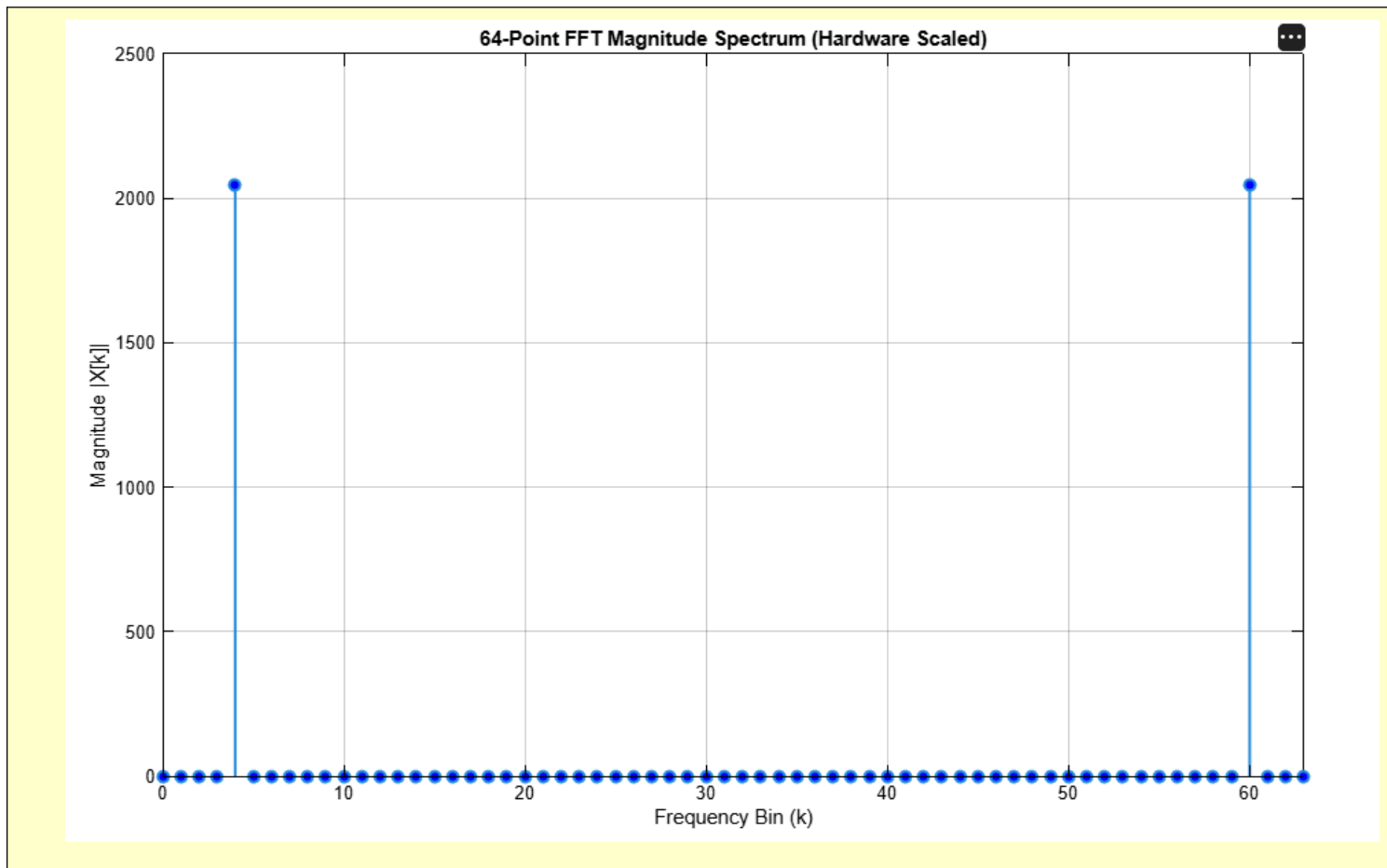
```
lXout_i2_temp = 2 * ( *(p_xa+1) << 15 ) - lXout_i1_temp;
```

Lab task 2: 64 point FFT

- Test the 64-point FFT with the following signal written directly to *asInBuf[]* and compare the result with that from MATLAB.

$x[n] = 1000 \cdot \sin(2\pi \cdot 400 \cdot n / 64) + (0.5)^n \cdot \sin(2\pi \cdot 60 \cdot n / 64)$

- Screenshots of MATLAB plot and graphical display window



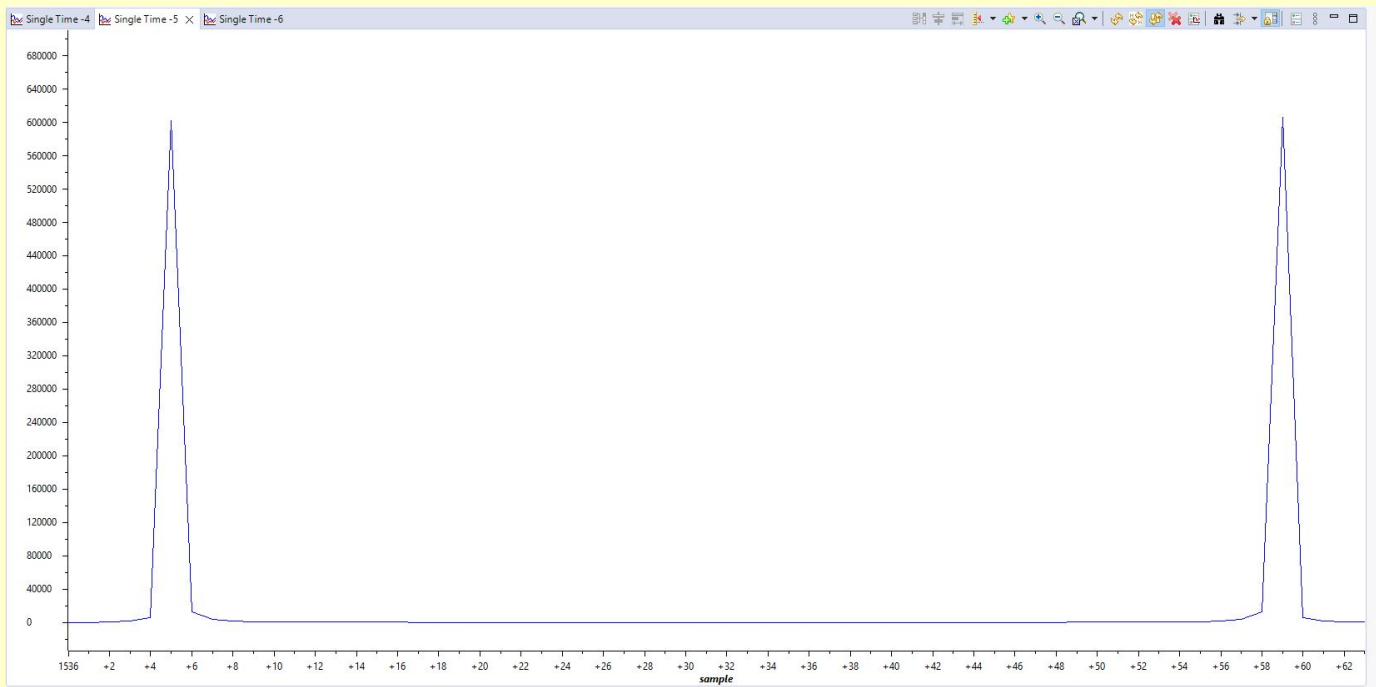
Lab task 3: Real-time spectrum analyser

Implement the analyzer according to the description of the algorithm above.

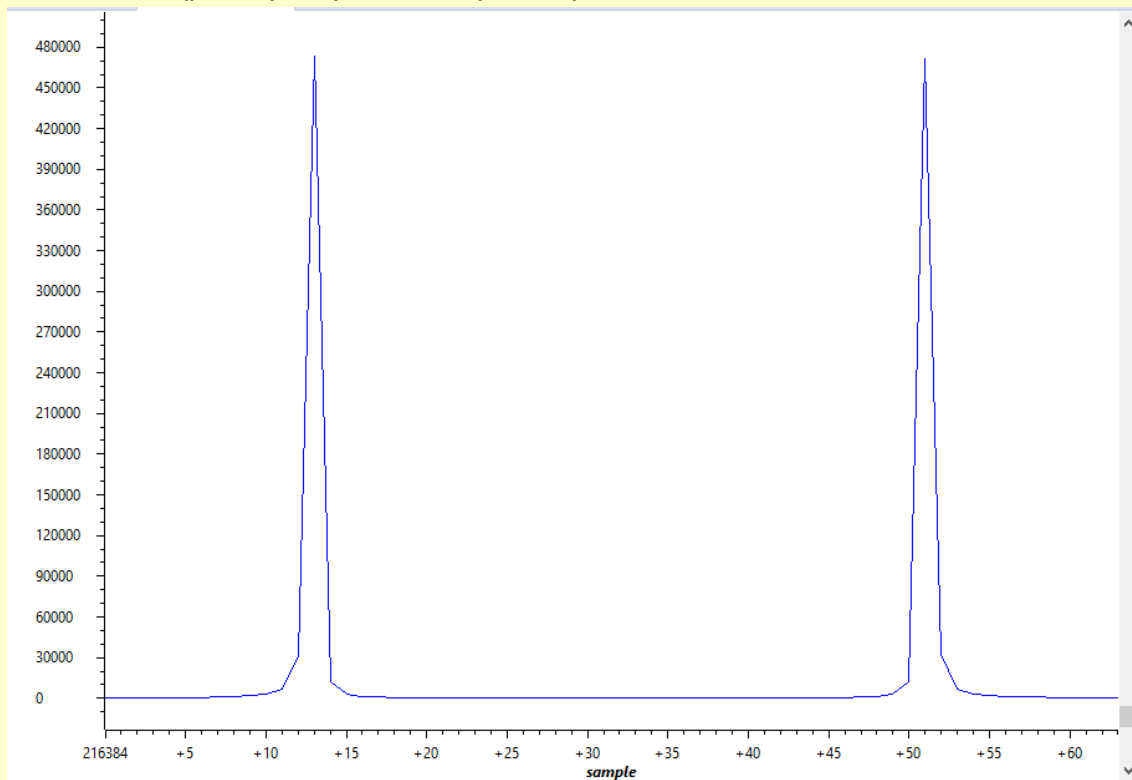
Verify that the FFT64_Analyser.c functions correctly:

Connect the signal generator to the DSK board and select "Waveform Sinus". Choose an amplitude of $2 V_{pp}$.

- Screenshot (part 1) (for 1kHz) (with 1Vpp for all measurements!)

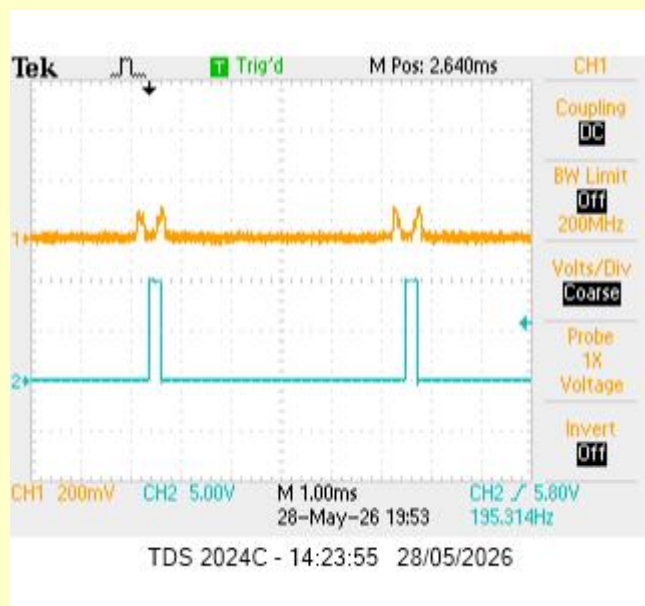


- Screenshot (part 2), explanation (15kHz)

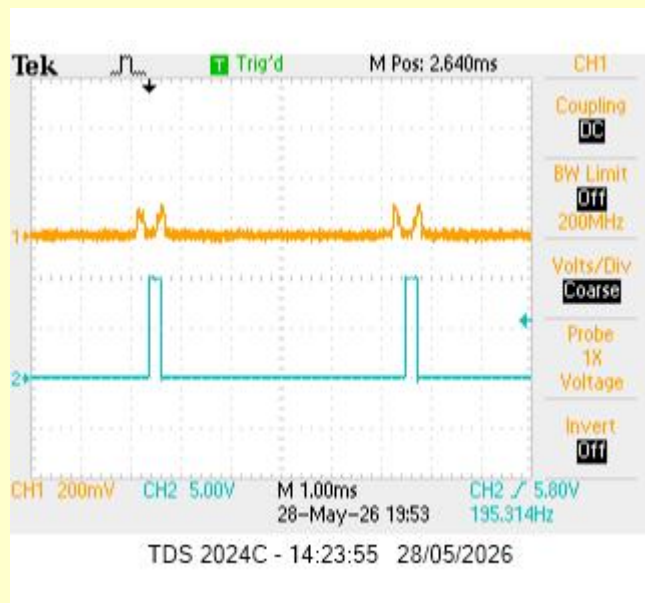


At $f_{in} = 15$ kHz, aliasing occurs because the input frequency is higher than the Nyquist frequency of 6.25 kHz. The signal is folded back to an alias frequency of approximately 2.5 kHz, so the FFT peak appears around bin $k = 13$.

- Two Oscilloscope screenshots (part 3)



500Hz



2kHz